And here as well, as always, we consider a binary partitioning "good" only if it manages to split the whole set being partitioned in two subsets approximately of the same size.

If we manage to obtain a good partitioning at each step, we will have a balanced tree with logarithmic height. Given a sequence of points it is, however, possible to choose an order of insertion for the points that will produce a skewed tree with linear height.

We'll see in a few pages how we can prevent this from happening, under certain conditions. Re-balancing on insertion is not, unfortunately, a viable option for k-d trees, as instead it was for binary search trees.

Before worrying about balancing trees, however, let's see in details the way methods like insert, remove, and all the queries work for k-d trees.

9.3 Methods

We have now seen a few examples of k-d trees, and how we can construct them: by now, it should be clear what a k-d tree looks like, the main ideas behind this data structure, and how we can leverage it.

It’s time to delve into the main methods for k-d trees, see how they work and how they can be implemented; in this section we will present pseudo-code for these methods, and you can find an actual implementation on our repo.